tests\test_utils_calvindate.py

```python
"""Tests for calvincTools.utils.calvindate module."""
import pytest
from datetime import datetime, date, timedelta
from calvincTools.utils.calvindate import calvindate, IsDateString


class TestCalvindateConstruction:
    """Test calvindate construction methods."""

    def test_construction_no_args(self):
        """Test construction with no arguments returns today."""
        cd = calvindate()
        today = datetime.today()
        assert cd.year == today.year
        assert cd.month == today.month
        assert cd.day == today.day

    def test_construction_year_month_day(self):
        """Test construction with year, month, day."""
        cd = calvindate(2024, 11, 15)
        assert cd.year == 2024
        assert cd.month == 11
        assert cd.day == 15

    def test_construction_month_day(self):
        """Test construction with month and day (current year)."""
        cd = calvindate(12, 25)
        current_year = date.today().year
        assert cd.year == current_year
        assert cd.month == 12
        assert cd.day == 25

    def test_construction_from_date_object(self):
        """Test construction from date object."""
        d = date(2024, 6, 15)
        cd = calvindate(d)
        assert cd.year == 2024
        assert cd.month == 6
        assert cd.day == 15

    def test_construction_from_datetime_object(self):
        """Test construction from datetime object."""
        dt = datetime(2024, 6, 15, 14, 30, 45)
        cd = calvindate(dt)
        assert cd.year == 2024
        assert cd.month == 6
        assert cd.day == 15
        assert cd.hour == 14
        assert cd.minute == 30
```

```
        assert cd.second == 45

    def test_construction_from_string(self):
        """Test construction from date string."""
        cd = calvindate("2024-11-15")
        assert cd.year == 2024
        assert cd.month == 11
        assert cd.day == 15

    def test_construction_from_string_various_formats(self):
        """Test construction from various date string formats."""
        formats = [
            "11/15/2024",
            "November 15, 2024",
            "15 Nov 2024",
            "2024-11-15"
        ]
        for fmt in formats:
            cd = calvindate(fmt)
            assert cd.year == 2024
            assert cd.month == 11
            assert cd.day == 15

    def test_construction_with_time(self):
        """Test construction with date and time components."""
        cd = calvindate(2024, 11, 15, 14, 30, 45)
        assert cd.year == 2024
        assert cd.month == 11
        assert cd.day == 15
        assert cd.hour == 14
        assert cd.minute == 30
        assert cd.second == 45

    def test_construction_with_microseconds(self):
        """Test construction with microseconds."""
        cd = calvindate(2024, 11, 15, 14, 30, 45, 123456)
        assert cd.microsecond == 123456


class TestCalvindateMethods:
    """Test calvindate methods."""

    def test_value(self):
        """Test value method returns self."""
        cd = calvindate(2024, 11, 15)
        assert cd.value() == cd

    def test_as_datetime(self):
        """Test as_datetime conversion."""
        cd = calvindate(2024, 11, 15, 10, 30, 45)
        dt = cd.as_datetime()
```

```
        assert isinstance(dt, datetime)
        assert dt.year == 2024
        assert dt.month == 11
        assert dt.day == 15
        assert dt.hour == 10
        assert dt.minute == 30

    def test_daysfrom_positive(self):
        """Test daysfrom with positive delta."""
        cd = calvindate(2024, 11, 15)
        future = cd.daysfrom(5)
        assert future.day == 20
        assert future.month == 11
        assert future.year == 2024

    def test_daysfrom_negative(self):
        """Test daysfrom with negative delta."""
        cd = calvindate(2024, 11, 15)
        past = cd.daysfrom(-5)
        assert past.day == 10
        assert past.month == 11
        assert past.year == 2024

    def test_daysfrom_cross_month(self):
        """Test daysfrom crossing month boundary."""
        cd = calvindate(2024, 11, 28)
        future = cd.daysfrom(5)
        assert future.day == 3
        assert future.month == 12
        assert future.year == 2024

    def test_tomorrow(self):
        """Test tomorrow method."""
        cd = calvindate(2024, 11, 15)
        tomorrow = cd.tomorrow()
        assert tomorrow.day == 16
        assert tomorrow.month == 11
        assert tomorrow.year == 2024

    def test_yesterday(self):
        """Test yesterday method."""
        cd = calvindate(2024, 11, 15)
        yesterday = cd.yesterday()
        assert yesterday.day == 14
        assert yesterday.month == 11
        assert yesterday.year == 2024

    def test_tomorrow_month_boundary(self):
        """Test tomorrow at month boundary."""
        cd = calvindate(2024, 11, 30)
        tomorrow = cd.tomorrow()
```

```
        assert tomorrow.day == 1
        assert tomorrow.month == 12

    def test_yesterday_month_boundary(self):
        """Test yesterday at month boundary."""
        cd = calvindate(2024, 12, 1)
        yesterday = cd.yesterday()
        assert yesterday.day == 30
        assert yesterday.month == 11


class TestCalvindateNextWorkday:
    """Test nextWorkdayAfter method."""

    def test_next_workday_from_friday(self):
        """Test next workday from Friday is Monday."""
        # December 6, 2024 is a Friday
        cd = calvindate(2024, 12, 6)
        next_wd = cd.nextWorkdayAfter()
        # Should be Monday, December 9
        assert next_wd.day == 9
        assert next_wd.month == 12
        assert next_wd.weekday() == 0  # Monday

    def test_next_workday_from_monday(self):
        """Test next workday from Monday is Tuesday."""
        # December 2, 2024 is a Monday
        cd = calvindate(2024, 12, 2)
        next_wd = cd.nextWorkdayAfter()
        # Should be Tuesday, December 3
        assert next_wd.day == 3
        assert next_wd.month == 12
        assert next_wd.weekday() == 1  # Tuesday

    def test_next_workday_include_after_date(self):
        """Test next workday with include_afterdate=True."""
        # December 2, 2024 is a Monday (workday)
        cd = calvindate(2024, 12, 2)
        next_wd = cd.nextWorkdayAfter(include_afterdate=True)
        # Should be the same day since Monday is a workday
        assert next_wd.day == 2
        assert next_wd.month == 12


class TestIsDateString:
    """Test IsDateString function."""

    def test_is_date_string_valid(self):
        """Test valid date strings."""
        valid_dates = [
            "2024-11-15",
```

```
            "11/15/2024",
            "November 15, 2024",
            "15 Nov 2024"
        ]
        for date_str in valid_dates:
            assert IsDateString(date_str) is True

    def test_is_date_string_invalid(self):
        """Test invalid date strings."""
        invalid_dates = [
            "not a date",
            "hello world",
            "12345",
            ""
        ]
        for date_str in invalid_dates:
            assert IsDateString(date_str) is False

    def test_is_date_string_edge_cases(self):
        """Test edge cases."""
        # Ambiguous but parseable
        assert IsDateString("1/1/1") is True
        # Invalid date
        assert IsDateString("2024-13-45") is False

```